SQL Exercise with Answers for Lecture 7

CS105.3 Database Management Systems

1. Using the `LIKE` Clause

Q1: Find all customers whose names start with the letter 'A'.

```
SELECT * FROM Customers

WHERE CustomerName LIKE 'A%';
```

A1: This query selects all customers whose names begin with 'A'. The '%' is a wildcard character that matches any sequence of characters.

Q2: Find all products with names that contain the word 'chair'.

```
SELECT * FROM Products
WHERE ProductName LIKE '%chair%';
```

A2: This query selects all products whose names contain the word 'chair' anywhere in the name. The '%' wildcard before and after 'chair' allows for any characters to precede or follow 'chair'.

Q3: Find all employees whose last name ends with 'son'.

```
SELECT * FROM Employees
WHERE LastName LIKE '%son';
```

A3: This query selects all employees whose last names end with 'son'. The '%' wildcard matches any sequence of characters before 'son'.

---

2. Using the `ORDER BY` Clause

Q4: Select all products and order them by price in ascending order.

```
SELECT * FROM Products
ORDER BY Price ASC;
```

A4: This query selects all products and orders them by their price in ascending order. `ASC` specifies ascending order.

Q5: Select all customers and order them by their registration date in descending order.

```
SELECT * FROM Customers
ORDER BY RegistrationDate DESC;
```

A5: This query selects all customers and orders them by their registration date in descending order. `DESC` specifies descending order.

Q6: Select all orders and order them by customer ID and then by order date within each customer ID.

```
SELECT * FROM Orders
ORDER BY CustomerID, OrderDate;
```

A6: This query orders the orders first by `CustomerID` and then by `OrderDate` within each `CustomerID`. The default order is ascending.

---

3. Using the `GROUP BY` Clause

Q7: Count the number of orders placed by each customer.

```
SELECT CustomerID, COUNT(*) AS OrderCount
FROM Orders
GROUP BY CustomerID;
```

A7: This query counts the number of orders for each customer. The `GROUP BY` clause groups rows by `CustomerID`, and the `COUNT(*)` function counts the rows in each group.

Q8: Find the average price of products in each category.

```
SELECT CategoryID, AVG(Price) AS AveragePrice
FROM Products
GROUP BY CategoryID;
```

A8: This query calculates the average price of products for each category. The `GROUP BY` clause groups rows by `CategoryID`, and the `AVG(Price)` function computes the average price in each group.

Q9: Find the total sales for each product.

```
SELECT ProductID, SUM(Quantity * UnitPrice) AS TotalSales
FROM OrderDetails
GROUP BY ProductID;
```

A9: This query calculates the total sales for each product. The `GROUP BY` clause groups rows by `ProductID`, and the `SUM(Quantity * UnitPrice)` function computes the total sales in each group.

---

4. Using the `WHERE` Clause

Q10: Select all products with a price greater than $20.

```
SELECT * FROM Products
WHERE Price > 20;
```

A10: This query selects all products with a price greater than $20. The `WHERE` clause filters rows based on the `Price` column.

Q11: Select all orders placed in the year 2023.

```
SELECT * FROM Orders
WHERE YEAR(OrderDate) = 2023;
```

A11: This query selects all orders placed in the year 2023. The `WHERE` clause filters rows where the year part of `OrderDate` is 2023.

Q12: Select all customers from the city 'New York'.

```
SELECT * FROM Customers
WHERE City = 'New York';
```

A12: This query selects all customers from the city 'New York'. The `WHERE` clause filters rows based on the `City` column.

---

5. Using the `HAVING` Clause

Q13: Find categories with an average product price greater than $50.

```
SELECT CategoryID, AVG(Price) AS AveragePrice
FROM Products
GROUP BY CategoryID
HAVING AVG(Price) > 50;
```

A13: This query groups products by `CategoryID`, calculates the average price for each group, and then filters the groups to include only those with an average price greater than $50.

Q14: Find customers who have placed more than 10 orders.

```
SELECT CustomerID, COUNT(*) AS OrderCount
FROM Orders
GROUP BY CustomerID
HAVING COUNT(*) > 10;
```

A14: This query groups orders by `CustomerID`, counts the number of orders for each customer, and then filters the groups to include only those with more than 10 orders.

Q15: Find products that have sold more than 100 units in total.

```
SELECT ProductID, SUM(Quantity) AS TotalUnitsSold
FROM OrderDetails
GROUP BY ProductID
HAVING SUM(Quantity) > 100;
```

A15: This query groups order details by `ProductID`, sums the quantity sold for each product, and then filters the groups to include only those with total units sold greater than 100.

---

6. Using the `IN` Clause

Q16: Select all customers from the cities 'New York', 'Los Angeles', or 'Chicago'.

```
SELECT * FROM Customers
WHERE City IN ('New York', 'Los Angeles', 'Chicago');
```

A16: This query selects all customers from 'New York', 'Los Angeles', or 'Chicago'. The `IN` clause filters rows to include only those with a `City` value matching one of the specified cities.

Q17: Select all employees who belong to departments 1, 2, or 3.

```
SELECT * FROM Employees
WHERE DepartmentID IN (1, 2, 3);
```

A17: This query selects all employees who belong to departments 1, 2, or 3. The `IN` clause filters rows to include only those with a `DepartmentID` value matching one of the specified IDs.

Q18: Select all products that belong to categories 5, 10, or 15.

```
SELECT * FROM Products
WHERE CategoryID IN (5, 10, 15);
```

A18: This query selects all products that belong to categories 5, 10, or 15. The `IN` clause filters rows to include only those with a `CategoryID` value matching one of the specified IDs.

---

7. Using the `BETWEEN` Clause

Q19: Select all orders placed between January 1, 2023, and March 31, 2023.

```
SELECT * FROM Orders
WHERE OrderDate BETWEEN '2023-01-01' AND '2023-03-31';
```

A19: This query selects all orders placed between January 1, 2023, and March 31, 2023. The `BETWEEN` clause filters rows based on the `OrderDate` column to include only those within the specified date range.

Q20: Select all products with prices between $10 and $50.

```
SELECT * FROM Products
WHERE Price BETWEEN 10 AND 50;
```

A20: This query selects all products with prices between $10 and $50. The `BETWEEN` clause filters rows to include only those with a `Price` value within the specified range.

Explanation of Clauses:

1. **LIKE:** Used for pattern matching with wildcards (`%` for multiple characters, _ for a single character).
2. **ORDER BY:** Sorts the result set by one or more columns in ascending (`ASC`) or descending (`DESC`) order.

3. **GROUP BY:** Groups rows sharing a property so that aggregate functions can be applied to each group.
4. **WHERE:** Filters rows based on a condition.
5. **HAVING:** Filters groups based on a condition, often used with `GROUP BY`.
6. **IN:** Checks if a value matches any value in a list.
7. **BETWEEN:** Selects values within a given range (inclusive).